Vibhas Tallapalli

vtallapa@uwaterloo.ca | vibhastallapalli.github.io | linkedin.com/in/vibhas-tallapalli | github.com/vibhastallapalli | +1 (647) 975-6479

Education

University of Waterloo — Waterloo, ON
BASc in Nanotechnology Engineering — *Sep 2025 – Present*

Technical Skills

Software: Python, C++, C, JavaScript, MATLAB, Arduino C++, SQL
Libraries/Frameworks: Flask, Playwright, pandas, Streamlit, Plotly, Chart.js, Tkinter, Matplotlib
Data/Databases: SQLite, normalized schema design, ETL pipelines, real time data logging
Quantitative/Finance: Black Scholes, Greeks (Delta, Gamma, Theta, Vega), Monte Carlo, Sharpe ratio, backtesting, PID control
Engineering/Hardware: SolidWorks, FEA, GD&T, Tolerance, MATLAB simulation, Embedded Systems, Sensor Integration (PPG, IMU)

Experience

Linear Automation Inc. | *SQL, Python, MATLAB, FEA, SolidWorks* — Barrie, ON
Mechanical Engineer Co-op — *Apr 2026 – Present*

- Designed and stress-tested **5+ structural Honda vehicle frame components** in **SolidWorks** using **MATLAB** and **FEA** across **7+ iterations**, sweeping wall thickness and load cases to isolate stress concentrations, then upgrading material (**Al 5000→6000 series**) to cut component weight by **30%** while maintaining **300+ lb** load requirements.
- Queried the **manufacturing execution system** in **SQL**, filtering by machine ID, shift, and rejection code to surface defect trends, flagged one machine causing most night-shift rejects, and escalated it; the resulting calibration fix reduced the plant-wide rejection rate by **22%**.
- Built a **Python** script that reads **SolidWorks** drawing feature types and auto-assigns decimal precision per internal **GD&T** standards, eliminating manual precision decisions across **20+ engineering drawings** and reducing precision review comments by **90%**.

IntegraYouth | *Outreach, Public Speaking, Leadership, Event Planning* — Toronto, ON
Secretary | Blog Writer | Executive — *June 2024 – Present*

- Wrote **50+ blog posts** on programs and impact, reaching **1,000+ views** and improving outreach through consistent publishing and topic planning.
- Ran Secretary-team operations and supported event logistics, communications, and scheduling for **5+ events** with **200+ attendees**.

Projects

Probable: On-Device AI Sports Wagering App | *Electron, React, Node.js, Qwen3 VL, llama.cpp* — May 2026

- Ran a **2B vision language model (Qwen3 VL) fully on device** through **llama.cpp** to rank two football goal clips, scoring each **out of 40** on a fixed rubric with **zero cloud inference**, inside a **peer-to-peer desktop app** (Electron, React, Node.js) where two players stake testnet USDt on the verdict.
- Designed a **three-pass judging pipeline** (describe each clip, locate the shot on a grid, then score both head to head in one call) that **matched human picks on 74.6%** of test pairs, fixing the identical scores separate grading produced.
- Built an evaluation harness that re-judges every labeled clip pair in both orders, reporting order-consistency (a positional-bias metric), human agreement, and per-dimension winners with Wilson 95% confidence intervals; a unit-tested metrics core sits outside the model, so results reproduce.

World Cup Yellow Card Predictor | *Python, statsmodels, Flask, StatsBomb data* — June 2026

- Built a **predictive model** that forecasts a match's yellow card total and **flags games where its probability beats the book's implied odds**, hitting on **10 of 13 calls (77%)**; **Negative Binomial fit** on **StatsBomb data(API)** across **530 World Cup matches**, with line priced from the full distribution.
- Found that the **referee, not the teams, is the strongest predictor** of a match's card count, so built **referee features** (yellows per foul, average first card minute, strictness percentile) from **165 referee profiles and 1,656 timestamped bookings**, with **Bayesian shrinkage** stabilizing sparse data.

AI Job Application Automation Bot | *Python, Flask, Playwright, SQLite, Anthropic API* — Jan 2026

- Built a **full-stack job-application bot** in **Python** (Flask, Playwright, BeautifulSoup, SQLite) that scrapes postings and auto-submits applications; answers form fields with the **Anthropic API grounded by a RAG layer**, retrieving only the background chunks relevant to each question from a **SQLite vector store** by **cosine similarity** to **curb hallucination** and cut tokens.
- Designed a **4-table normalized SQLite schema** (jobs, applications, resumes, logs) tracking **6 application stages** with **applicant count filtering** to target **low-competition postings**, surfaced through a **real-time Flask dashboard**, cutting per-application time from **15 minutes to 30 seconds**.

Vouch: Zero-Knowledge Rental Applications | *Compact, Midnight blockchain, Node.js, React* — July 2026

- Built and compiled a **zero-knowledge proof** (**Compact, Midnight blockchain**) that lets a renter **prove they make at least 3x the rent without revealing their income, ID, or pay stubs**, so a landlord screens on whether someone qualifies, not who they are. Circuit is **138 lines** and compiles to a **5.2 MB prover and 2 KB verifier**.
- Made every applicant **anonymous until a landlord picks one winner**, and blocked **fake duplicate applications** with a **commitment-and-nullifier scheme** (the anti-double-spend trick behind privacy coins) that **caps each person to one application per listing** while keeping **only hashes** on chain, no name and no income.